

Module 8 Lab Session 2: Components, Lists, and Routing Spine

Module	M8 Angular 22 Fundamentals
Session	2 of 3
Exercices	Exercices 2 , 3 , and 4

Story thread

You move from a single dashboard into composable UI: course cards, lists with honest empty states, and a **real route** to course detail. **Do not skip Lab 4** before Session 3 forms and HTTP Exercices expect a coherent router tree.

Exercise 2 Breaking the Monolith: Component Communication

Your dashboard should not contain 500 lines of HTML for every feature. You need a CourseCardComponent that knows how to display one course and tell the parent when the user clicks "Enroll."

Step 1: Generate the Card Component

ng generate component ui/course-card

Step 2: Define the Component Contract

Open course-card.component.ts:

```
import { Component, input, output } from "@angular/core";
import { Course } from "../../models/course.model";

@Component({
  selector: "tms-course-card",
  standalone: true,
  templateUrl: "./course-card.component.html",
  styleUrls: ["./course-card.component.scss"],
})
export class CourseCardComponent {
  course = input.required<Course>();
  enrollClicked = output<Course>();
}
```

Two new concepts here:

- **input.required<Course>()** This declares that the parent component *must* pass a Course object to this child. The <Course> part is a TypeScript **generic** it tells Angular the type of data expected. If the parent forgets to pass the course, Angular throws a compile-time error, not a runtime crash. Think of it like a required parameter on a C# method.
- **output<Course>()** This declares that this child component can send a Course event back up to the parent. The parent listens for this event the same way you listen for a button click. You will see how to wire it in Step 4.

Notice the selector: 'tms-course-card' this means you will use <tms-course-card> as the HTML tag in the parent's template. You can name this anything, but prefixing with tms- makes it clear this is a TMS component, not a built-in HTML tag.

Step 3: Build the Card Template

Open course-card.component.html:

```
<div class="card">
  <h3>{{ course().title }} ({{ course().code }})</h3>
  <p>
    Enrolled {{ course().enrollmentCount }} of {{ course().maxCapacity }}
    seats
  </p>
  <span
    class="badge"
    [class.closed]="course().enrollmentCount >= course().maxCapacity"
  >
    {{ course().enrollmentCount >= course().maxCapacity ? "Full" : "Accepting
    enrollments" }}
  </span>
  <button
    (click)="enrollClicked.emit(course())"
    [disabled]="course().enrollmentCount >= course().maxCapacity"
  >
    Enroll
  </button>
</div>
```

New Angular syntax in this template:

- **course().title** Because course is an input() signal, you call it with () to get the value, then access .title on the resulting Course object.

- `[class.closed]="course().enrollmentCount >= course().maxCapacity"`
The square brackets `[ ]` mean this is a **property binding**. When the class is full, Angular adds the CSS class `closed`. This mirrors the capacity fields on the **TMS API contract** list row (`enrollmentCount`, `maxCapacity`) that your catalogue API already exposes.
- `(click)="enrollClicked.emit(course())"` When the button is clicked, this calls `.emit()` on the output you defined, sending the current course object up to the parent.
- `[disabled]="course().enrollmentCount >= course().maxCapacity"`
Another property binding. When the course is full, the button becomes disabled (greyed out, unclickable).

Step 4: Use the Card in the Dashboard

Go back to `student-dashboard.component.ts`. You need to do three things: import the card component, add mock course data, and write a handler for the enroll event.

First, add these imports at the top of the file:

```
import { CourseCardComponent } from "../../ui/course-card/course-card.component";
import { Course } from "../../models/course.model";
```

Then, update the `@Component` decorator to include the child component. Add `CourseCardComponent` to the imports array:

```
@Component({
  selector: 'app-student-dashboard',
  standalone: true,
  imports: [CourseCardComponent], // This tells Angular: "I use Course CardComponent in my template"
  templateUrl: './student-dashboard.component.html',
  styleUrls: ['./student-dashboard.component.scss']
})
```

Why is this needed? Because standalone components manage their own dependencies. If you use `<tms-course-card>` in the template but do not add it to imports, Angular does not know what that tag is it silently renders it as text in the DOM. No error. No warning. Just nothing where the card should be.

Now add a sample course and a handler to the class body:

```
// signal<Course | null>(null) means: "This signal holds either a Course or nothing."
// The | null syntax is TypeScript's way of saying a value can be absent.
```

```

selectedCourse = signal<Course | null>(null);

// A sample course to display (we will switch to an array in Exercise
3)
sampleCourse: Course = {
  id: 1,
  title: "Advanced Java Services",
  code: "CSE-101",
  maxCapacity: 30,
  enrollmentCount: 12,
};

handleEnroll(course: Course) {
  this.selectedCourse.set(course);
  console.log('Enrollment requested for:', course.title);
}

```

In student-dashboard.component.html, add below the existing dashboard content:

```

<h2>Available Courses</h2>
<tms-course-card
  [course]="sampleCourse"
  (enrollClicked)="handleEnroll($event)"
/>

```

Here is what each part of the HTML means:

- **[course]="sampleCourse"** Square brackets = **property binding**. This passes the sampleCourse object from the parent into the child's course input. It is like passing a parameter to a function.
- **(enrollClicked)="handleEnroll(\$event)"** Parentheses = **event binding**. When the child component calls enrollClicked.emit(...), Angular runs handleEnroll(...) in the parent. The \$event is a special Angular variable that contains whatever the child emitted in this case, the Course object.

Checkpoint 2

- ☐ The course card renders inside the dashboard
- ☐ Clicking "Enroll" logs the course title to the browser console
- ☐ The "Enroll" button is disabled when the course is full (enrollmentCount >= maxCapacity)

Note: selectedCourse updates in memory in this lab, but the template does not show it yet that is intentional. **Exercise 3** adds a visible "Last enrollment request" line so Enroll clicks change the screen, not only the console.

Exercise 3 Loops, Conditionals, and the Empty State

You have one card working. Now you need a catalog of courses, and you need to handle the case where there are no courses available.

Step 1: Create the Catalog Data

In `student-dashboard.component.ts`, replace or add:

```
availableCourses = signal<Course[]>([
  {
    id: 1,
    title: "Advanced Java Services",
    code: "CSE-101",
    maxCapacity: 30,
    enrollmentCount: 10,
  },
  {
    id: 2,
    title: "Angular UI Lab",
    code: "CSE-210",
    maxCapacity: 25,
    enrollmentCount: 25,
  },
  {
    id: 3,
    title: "Database Design",
    code: "CSE-305",
    maxCapacity: 20,
    enrollmentCount: 18,
  },
  {
    id: 4,
    title: "API Security Workshop",
    code: "CSE-420",
    maxCapacity: 40,
    enrollmentCount: 15,
  },
]);
```

Step 2: Render the Loop

Cleanup from Exercise 2 (required): Exercise 2 left a **single** `<tms-course-card>` bound to `sampleCourse` under a heading like “Available Courses.” If you leave that in place, you will see **one hardcoded card plus four loop cards** that is not a bug in Angular; it is leftover markup. **Remove** the Exercise 2-only block: the extra `<h2>` and the single `<tms-course-card [course]="sampleCourse" ... />`. You can delete the `sampleCourse` property from the class once nothing

references it; keep `selectedCourse` and `handleEnroll` for the loop's (`enrollClicked`) bindings.

In the template, after cleanup, use something like:

```
<h2>Course Catalog</h2>

@if (availableCourses().length === 0) {
  <div class="empty-state">
    <p>No courses are available this term. Check back during registration.
  </p>
  </div>
} @else {
  <div class="grid">
    @for (course of availableCourses(); track course.id) {
      <tms-course-card [course]="course" (enrollClicked)="handleEnroll($event)" />
    } @empty {
      <p>No results match your search.</p>
    }
  </div>
} @if (selectedCourse(); as picked) {
  <p class="selection-hint" role="status">
    Last enrollment request: <strong>{{ picked.title }}</strong> ({{ picked.ed.code }})
  </p>
}
```

Use `selectedCourse()` in the template it is a **signal**, so the parentheses call the current value.

Why this @if block: In Exercise 2, `handleEnroll` only called `console.log`. Beginners often think the UI is “broken” because nothing on the page changes. This line proves the parent received the event no DevTools required. You can style `.selection-hint` in the component SCSS (for example subtle border or background).

Two things to pay attention to:

1. **track course.id** This is mandatory. It tells Angular which DOM node belongs to which course. If you omit `track`, the application will not compile.
2. **@empty** This is a fallback that renders if the array is specifically empty *after* the loop starts. It is different from the outer `@if` check. In practice you will usually use one or the other depending on whether you want to show

different messages for “no data loaded yet” versus “data loaded but filtered to zero results.”

Step 3: Test the Empty State

Temporarily set `availableCourses` to an empty array: `signal<Course[]>([])`. The empty state message should appear. Then restore the data.

Checkpoint 3

- ☐ Only the **catalog** cards appear (no duplicate single `sampleCourse` card above the grid)
- ☐ Four course cards render in a grid
- ☐ The full course (“Angular UI Lab”) has a disabled Enroll button
- ☐ Clicking **Enroll** on a course updates the on-page “Last enrollment request” line (not only the console)
- ☐ Setting the array to empty shows the empty state message

Exercise 4 Routing to a Course Detail Page

When Liya clicks on a course title, she should navigate to `/courses/1` and see proof that the **route parameter** arrived (this Exercise is the **navigation spine**). A rich detail screen (title, seat counts, HATEOAS links from `CourseDetailDto`) would use `CourseService.getById(id())` or data you already have; that layering is intentionally **out of scope** here so you are not debugging **HTTP and routing** in the same hour.

Step 1: Generate the Detail Component

ng generate component features/course-detail

Step 2: Add the Route

In `app.routes.ts`, add:

```
{
  path: 'courses/:id',
  loadComponent: () => import('./features/course-detail/course-detail.component')
    .then(m => m.CourseDetailComponent)
}
```

Step 3: Use Input Binding for the Route Parameter

In `course-detail.component.ts`:

```

import { Component, input, effect } from "@angular/core";

@Component({
  selector: "app-course-detail",
  standalone: true,
  templateUrl: "./course-detail.component.html",
})
export class CourseDetailComponent {
  // This automatically receives the :id from the URL /courses/:id
  // Because we enabled withComponentInputBinding() in app.config.ts (Step 2 of Exercise 1),
  // Angular maps the URL parameter ":id" directly to this input.
  // The name must match exactly: the route says ":id", so the input is called "id".
  id = input.required<string>();

  // The constructor runs when the component is created.
  // effect() watches any signals read inside it. Every time id() changes
  // (e.g. navigating from /courses/1 to /courses/2), this code runs again.
  constructor() {
    effect(() => {
      console.log(`Loading course detail for ID: ${this.id()}`);
    });
  }
}

```

In the template:

```

<h1>Course Detail</h1>
<p>Course ID: {{ id() }}</p>
<a routerLink="/dashboard">Back to Dashboard</a>

```

The routerLink attribute tells Angular: "When the user clicks this link, navigate to /dashboard without reloading the entire page." This is different from a normal HTML `<a href="/dashboard">` a normal link would reload the whole app. Angular's routerLink does a soft navigation, keeping the app state intact.

To use routerLink, you must import RouterLink in the component:

```

import { RouterLink } from '@angular/router';

@Component({
  // ... existing properties
  imports: [RouterLink] // Add this
})

```


Step 4: Link from the Course Card

Back in `course-card.component.html`, wrap the course title in a link that navigates to the detail page:

```
<h3>
  <a [routerLink]="['/courses', course().id]"> {{ course().title }} </a>
  ({{ course().code }})
</h3>
```

The `[routerLink]="['/courses', course().id]"` builds a URL dynamically. If `course().id` is 1, Angular navigates to `/courses/1`. The square brackets around `routerLink` mean this is a property binding Angular evaluates the expression, it is not a static string.

You also need to add `RouterLink` to `CourseCardComponent`'s `imports` array, just like you did for the detail component.

Troubleshooting

Problem	Cause	Fix
Clicking the link does nothing	<code>RouterLink</code> not imported in the component	Add <code>RouterLink</code> to the <code>imports</code> array
URL changes but page is blank	Missing <code><router-outlet /></code> in <code>app.component.html</code>	Ensure the outlet exists
<code>id()</code> returns undefined	Input name does not match route parameter name	The route says <code>:id</code> , so the input must be called <code>id</code>

Checkpoint 4

- ☐ Clicking a course title navigates to `/courses/1` (or whatever the ID is)
- ☐ The detail page displays the correct course ID
- ☐ The "Back to Dashboard" link works